

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;

public class EmployeeBonus {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter employee salary: ");
        double salary = sc.nextDouble();

        System.out.print("Enter years of service: ");
        int years = sc.nextInt();

        System.out.print("Enter performance rating (1.0 - 5.0): ");
        double rating = sc.nextDouble();

        double bonusPercent = 0.0;

        if (rating < 3.0) {
            bonusPercent = 0.0;
        }
        else {
            if (years < 3) {
                bonusPercent = 0.0;
            }
            else if (years >= 3 && years <= 5) {
                bonusPercent = 5.0;
            }
            else if (years >= 6 && years <= 10) {
                bonusPercent = 10.0;
            }
            else if (years > 10) {
                bonusPercent = 15.0;
            }

            if (rating >= 4.5) {
                bonusPercent += 2.0;
            }
        }
    }
}
```

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

```
double bonusAmount = (salary * bonusPercent) / 100;

System.out.println("\nBonus Percentage: " + bonusPercent + "%");
System.out.println("Bonus Amount: " + bonusAmount);
System.out.println("Total Salary (with bonus): " + (salary + bonusAmount));
    }
}
```

RUBRIC

SL	Points To Meet	Marks [10]
1	Proper import and java syntax	1
2	Creating scanner object and taking inputs and prompts properly	1
3	Proper bonus percentage logic	5
4	Checking for extra bonus	2
5	Proper output	1

[SET-B]

SOLUTION

CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics

```
import java.util.Scanner;

public class SalesCommission {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total sales amount: ");
        double sales = sc.nextDouble();

        System.out.print("Enter average customer satisfaction rating (1.0 - 5.0): ");
        double rating = sc.nextDouble();

        double commissionPercent = 0.0;

        if (rating < 2.5) {
            commissionPercent = 0.0;
        }
        else {
            if (sales < 50000) {
                commissionPercent = 0.0;
            }
            else if (sales >= 50000 && sales <= 99999) {
                commissionPercent = 5.0;
            }
            else if (sales >= 100000 && sales <= 200000) {
                commissionPercent = 10.0;
            }
            else if (sales > 200000) {
                commissionPercent = 15.0;
            }

            if (rating >= 4.0) {
                commissionPercent += 2.0;
            }
        }

        double commissionAmount = (sales * commissionPercent) / 100;

        System.out.println("Commission Percentage: " + commissionPercent + "%");
    }
}
```

CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics

```
        System.out.println("Commission Amount: $" + commissionAmount);
        System.out.println("Total Payout (Sales + Commission): $" + (sales +
commissionAmount));
    }
}
```

RUBRIC

SL	Points To Meet	Marks [10]
1	Proper import and java syntax	1
2	Creating scanner object and taking inputs and prompts properly	1
3	Proper commission percentage logic	5
4	Checking for extra commission	2
5	Proper output	1

Name :	ID:	Section:
--------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program that calculates the bonus amount for an employee based on their years of service and performance rating. The program should prompt the user to input three values: the employee's salary, their total years of service in the company, and their performance rating (on a scale from 1.0 to 5.0).

The bonus percentage should be determined according to the following criteria:

- Employees with **less than 3 years** of service receive **no bonus (0%)**.
- Employees with **3 to 5 years** of service receive a **5% bonus**.
- Employees with **6 to 10 years** of service receive a **10% bonus**.
- Employees with **more than 10 years** of service receive a **15% bonus**.

Additionally, if an employee's performance rating is **4.5 or higher**, they should receive an **extra 2% bonus** on top of their base bonus. However, if the performance rating is **below 3.0**, the employee is **not eligible for any bonus**, regardless of their years of service. Your program should correctly calculate and display the total bonus amount based on these rules.

Sample Input	Sample Output
Enter employee salary: 50000 Enter years of service: 7 Enter performance rating (1.0 - 5.0): 4.6	Bonus Percentage: 12.0% Bonus Amount: 6000.0 Total Salary (with bonus): 56000.0
Enter employee salary: 40000 Enter years of service: 2 Enter performance rating (1.0 - 5.0): 4.8	Bonus Percentage: 0.0% Bonus Amount: 0.0 Total Salary (with bonus): 40000.0

Name :	ID:	Section:
--------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program that calculates the **sales commission** for a salesperson based on their **total sales amount** and **customer satisfaction rating**. The program should prompt the user to input two values: their total sales amount, and their average customer satisfaction rating (on a scale from 1.0 to 5.0).

The commission percentage should be determined according to the following criteria:

- Sales less than 50,000 → 0% commission
- Sales between 50,000 and 99,999 → 5% commission
- Sales between 100,000 and 200,000 → 10% commission
- Sales greater than 200,000 → 15% commission

Additionally If the customer satisfaction rating is **4 or higher**, the salesperson receives an **extra 2% commission** on top of their base commission. If the rating is **below 2.5**, the salesperson is **not eligible for any commission**, regardless of their total sales.

Your program should calculate and display the **total commission percentage**, the **commission amount**, and the **final payout** (sales amount + commission) based on these rules.

Sample Input	Sample Output
Enter total sales amount: 120000 Enter average customer satisfaction rating (1.0 - 5.0): 4.5	Commission Percentage: 7.0% Commission Amount: \$4200.0 Total Payout (Sales + Commission): \$64200.0
Enter total sales amount: 45000 Enter average customer satisfaction rating (1.0 - 5.0): 3.8	Commission Percentage: 0.0% Commission Amount: \$0.0 Total Payout (Sales + Commission): \$45000.0

CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;

public class DataUsageTracker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter N: ");
        int N = sc.nextInt();

        int dataScore = 0;

        for (int i = 1; i <= N; i++) {
            System.out.print("Enter data for day " + i + ": ");
            int data = sc.nextInt();

            if (data < 50 || data > 5000) {
                System.out.println("Invalid data amount, " + data + ".");
                i--;
                continue;
            }

            if (data % 4 == 0 || data % 10 == 5) {
                dataScore += data;
            } else {
                dataScore -= data;
            }
        }

        int scoreCopy = dataScore;
        if (scoreCopy < 0) scoreCopy *= -1;

        int evenCount = 0;
        while (scoreCopy > 0) {
            int digit = scoreCopy % 10;
            if (digit % 2 == 0) evenCount++;
            scoreCopy /= 10;
        }

        System.out.println("Final Data Score: " + dataScore);
        System.out.println("Number of even digits: " + evenCount);
    }
}
```


[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

RUBRIC

SL	Points To Meet	Marks [10]
1	Scanner used properly, N input taken	1.5
2	Loop runs N times with re-prompt	1
3	Proper Range check (50–5000)	1.5
4	Score updated using rules	2.5
5	Even-digit counting + negative handling	2.5
6	Output displayed correctly	1

[SET-B]

SOLUTION

```
import java.util.Scanner;

public class WaterConsumptionTracker {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter N: ");
        int N = sc.nextInt();

        int hydrationScore = 0;

        for (int i = 1; i <= N; i++) {
            System.out.print("Enter water for day " + i + ": ");
            int water = sc.nextInt();

            if (water < 100 || water > 10000) {
                System.out.println("Invalid water amount, " + water + ".");
                i--; // re-prompt this day
                continue;
            }

            if (water % 5 == 0 || water % 10 == 2) {
                hydrationScore += water;
            } else {
                hydrationScore -= water;
            }
        }

        int scoreCopy = hydrationScore;
        if (scoreCopy < 0) scoreCopy *= -1;

        int oddCount = 0;
        while (scoreCopy > 0) {
            int digit = scoreCopy % 10;
            if (digit % 2 != 0) oddCount++;
            scoreCopy /= 10;
        }

        System.out.println("Final Hydration Score: " + hydrationScore);
        System.out.println("Number of odd digits: " + oddCount);
    }
}
```

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

RUBRIC

SL	Points To Meet	Marks [10]
1	Scanner creation & input N taken	1.5
2	Loop runs N times, reprompt on invalid input	1
3	Range check 100–10,000 ml	1.5
4	Score updated based on divisible by 5 or ends with 2 rule	2.5
5	Odd digit counting with negative handling	2.5
6	Final output formatting correct	1

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A university student records the amount of mobile data (in MB) they use each day for **N valid days**. A daily data value is **invalid** if it is less than **50 MB** or greater than **5000 MB**.

For invalid inputs, print:

Invalid data amount, {value}.

Re-ask for that day until a valid amount is entered. After collecting N valid amounts, compute the **Data Score**:

- **Add** the data value if
 - it is divisible by 4, OR
 - it ends with digit 5
- **Otherwise subtract** the data value

Finally:

1. Print the **Final Data Score**
2. Count and print the number of **even digits** in the score
 - If negative, convert to positive before counting

Sample Input	Sample Output
Enter N: 5 Enter data for day 1: 120 Enter data for day 2: 40 Enter data for day 2: 144 Enter data for day 3: 3000 Enter data for day 4: 5020 Enter data for day 4: 900 Enter data for day 5: 425	Invalid data amount, 40. Invalid data amount, 5020. Final Data Score: 4589 Number of even digits: 2

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A university student tracks the amount of water (in milliliters) they drink each day for **N valid days**. A daily water amount is **invalid** if it is less than **100 ml** or greater than **10,000 ml**.

For invalid inputs, print:

Invalid water amount, {value}.

Re-ask for that day until a valid amount is entered. After collecting **N valid amounts**, compute a **Hydration Score** as follows:

- **Add** the water amount if
 - it is divisible by 5, OR
 - it ends with digit 2
- **Otherwise subtract** the water amount

Finally:

1. Print the **Final Hydration Score**
2. Count and print the number of **odd digits** in the score
 - If negative, convert to positive before counting

Sample Input	Sample Output
Enter N: 6 Enter water for day 1: 500 Enter water for day 2: 250 Enter water for day 3: 95 Enter water for day 3: 402 Enter water for day 4: 720 Enter water for day 5: 1234 Enter water for day 6: 832	Invalid water amount, 95. Final Hydration Score: 1470 Number of odd digits: 2

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;

public class PositiveImpactScore {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your message: ");
        String input = sc.nextLine();

        int vowels = 0;
        int consonants = 0;

        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);

            if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
            {
                if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O'
                || ch == 'U' ||
                    ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o'
                || ch == 'u') {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        int totalScore = vowels * 2 + consonants * 1;

        if (totalScore >= 20) {
            System.out.println("Highly Positive");
        } else if (totalScore >= 10) {
            System.out.println("Somewhat Positive");
        } else {
            System.out.println("Needs More Positivity");
        }
    }
}
```

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

RUBRIC

SL	Points To Meet	Marks [10]
1	Java syntax correct, proper import of Scanner class, and creating Scanner object	1
2	Taking proper input with prompts as shown in sample input/output	1
3	Correctly identifying vowels and consonants using loop	2
4	Correctly ignoring non-alphabet characters (digits, spaces, symbols)	1
5	Accurate calculation of total score according to the rules	3
6	Correct classification of the score into categories (Highly/Moderately/Needs More)	1.5
7	Proper print statements matching the sample output	0.5

[SET-B]

SOLUTION

```
import java.util.Scanner;

public class EnergyScore {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your message: ");
        String input = sc.nextLine();

        double vowels = 0;
        double consonants = 0;

        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);

            if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
            {
                if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O'
                || ch == 'U' ||
                    ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o'
                || ch == 'u') {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        double totalScore = vowels * 3 + consonants * 0.5;

        if (totalScore >= 50) {
            System.out.println("Highly Energetic");
        } else if (totalScore >= 25) {
            System.out.println("Moderately Energetic");
        } else {
            System.out.println("Needs More Energy");
        }
    }
}
```


[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

RUBRIC

SL	Points To Meet	Marks [10]
1	Java syntax correct, proper import of Scanner class, and creating Scanner object	1
2	Taking proper input with prompts as shown in sample input/output	1
3	Correctly identifying vowels and consonants using loop	2
4	Correctly ignoring non-alphabet characters (digits, spaces, symbols)	1
5	Accurate calculation of total score according to the rules	3
6	Correct classification of the score into categories (Highly/Moderately/Needs More)	1.5
7	Proper print statements matching the sample output	0.5

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are given a string as input that may contain uppercase letters, lowercase letters, spaces, digits, and special characters. Write a Java program to calculate the **Positive Impact Score** of the message using the rules below:

- Add **2 points** for every **vowel** (in both uppercase and lowercase)
- Add **1 point** for every **consonant** (in both uppercase and lowercase)
- **Ignore** all non-alphabet characters (digits, spaces, symbols)

After calculating the total score, print:

"**Highly Positive**" if the score is **20 or more**.

"**Somewhat Positive**" if the score is **between 10 and 19 (inclusive)**

"**Needs More Positivity**" if the score is **less than 10**.

You cannot use built-in methods other than `length()` and `charAt()`

Sample Case 1	Sample Case 2
User Input: Eat Sleep Code Repeat	User Input: hmm
Expected Output: Highly Positive	Expected Output: Needs More Positivity
Explanation: Vowels: $9 \times 2 = 18$ Consonants: $9 \times 1 = 9$ Total Score = $18 + 9 = 27$ points → Highly Positive	Explanation: Vowels: $0 \times 2 = 0$ Consonants: $3 \times 1 = 3$ Total Score= 3 points → Needs More Positivity

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are given a string as input that may contain uppercase letters, lowercase letters, spaces, digits, and special characters. Write a Java program to calculate the **Energy Score** of the message using the rules below:

- Add **0.5 point** for every **consonant** (in both uppercase and lowercase)
- Add **3 points** for every **vowel** (in both uppercase and lowercase)
- **Ignore** all non-alphabet characters (digits, spaces, symbols)

After calculating the total score, print:

"**Highly Energetic**" if the score is **50 or more**

"**Moderately Energetic**" if the score is **between 25 and 49** (inclusive)

"**Needs More Energy**" if the score is **less than 25**

You cannot use built-in methods other than `length()` and `charAt()`

Sample Case 1	Sample Case 2
User Input: Gaming, memes, coding, and pizza keep me awake all night	User Input: zzZZ
Expected Output: Highly Energetic	Expected Output: Needs More Energy
Explanation: Vowels = $17 \times 3 = 51$ Consonants = $27 \times 0.5 = 13.5$ Total Energy Score: $51 + 13.5 = 64.5$ points → Highly Energetic	Explanation: Vowels: $0 \times 3 = 0$ Consonants: $4 \times 0.5 = 2$ Total Score = 2 points → Needs More Energy

CSE110 Spring 2024 Lab Exam - 03 Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

[SET-A]

SOLUTION

```
import java.util.Scanner;

public class SecondMaxPairSum {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Initialize max and second max with first two pairs
        int maxSum = arr[0] + arr[1];
        int secondMaxSum = arr[0] + arr[2];

        // Swap if needed
        if (secondMaxSum > maxSum) {
            int temp = maxSum;
            maxSum = secondMaxSum;
            secondMaxSum = temp;
        }

        // Loop through all valid pairs
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                int sum = arr[i] + arr[j];

                if (sum > maxSum) {
                    secondMaxSum = maxSum;
                    maxSum = sum;
                } else if (sum > secondMaxSum && sum < maxSum) {
                    secondMaxSum = sum;
                }
            }
        }

        // Output
        System.out.println("Second Maximum pair sum: " + secondMaxSum);
    }
}
```

[SET-B]

SOLUTION

```
import java.util.Scanner;
public class SecondMinPairSum {

    // Method to find the second minimum pair sum
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Initialize with the first two pairs
        int minSum = arr[0] + arr[1];
        int secondMinSum = arr[0] + arr[2];

        // Ensure minSum < secondMinSum
        if (secondMinSum < minSum) {
            int temp = minSum;
            minSum = secondMinSum;
            secondMinSum = temp;
        }

        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                int sum = arr[i] + arr[j];

                if (sum < minSum) {
                    secondMinSum = minSum;
                    minSum = sum;
                } else if (sum > minSum && sum < secondMinSum) {
                    secondMinSum = sum;
                }
            }
        }

        System.out.println("Second Minimum pair sum: " +
secondMinSum);
    }
}
```

[CSE110 Spring 2024 Lab Exam - 0X Tentative Solutions and Rubrics](#)

RUBRIC

SL	Points to meet	Marks (10)
	Proper Java Syntax	1
	Correctly take user input	1.5
	Initialization of max/min and second max/min	1.5
	Correct nested loop logic	2
	Correct sum calculation	1
	Updating second max/min correctly	3
Total		10

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

Write a Java program called **SecondMaxPairSum** that takes an integer array with **unique values** as an user input. The program should **calculate all possible pairs (a, b)** such that **a comes before b** in the array, and prints the **second maximum pair sum** among these pairs. [You can assume the array contains at least three elements and all values of the array are unique.]

Sample Input	Sample Output	Explanation
int[] arr= {3,7,1,9,4};	Second Maximum pair sum: 13	From the array {3, 7, 1, 9, 4}, all pairs (a, b) where a comes before b are (3,7), (3,1), (3,9), (3, 4), (7, 1), (7, 9), (7, 4), (1, 9), (1, 4), (9, 4). Their sums are: 10, 4, 12, 7, 8, 16, 11, 10, 5, 13. The second maximum pair sum among them is 13.

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

Write a Java program called **SecondMinPairSum** that takes an integer array with **unique values** as an user input. The program should **calculate all possible pairs (a, b)** such that **a comes before b** in the array, and prints the **second minimum pair sum** among these pairs. [You can assume the array contains at least three elements and all values of the array are unique.]

Sample Input	Sample Output	Explanation
<pre>int[] arr= {3,7,1,9,4};</pre>	Second Minimum pair sum: 5	From the array {3, 7, 1, 9, 4}, all pairs (a, b) where a comes before b are (3,7), (3,1), (3,9), (3, 4), (7, 1), (7, 9), (7, 4), (1, 9), (1, 4), (9, 4). Their sums are: 10, 4, 12, 7, 8, 16, 11, 10, 5, 13. The second minimum pair sum among them is 5.